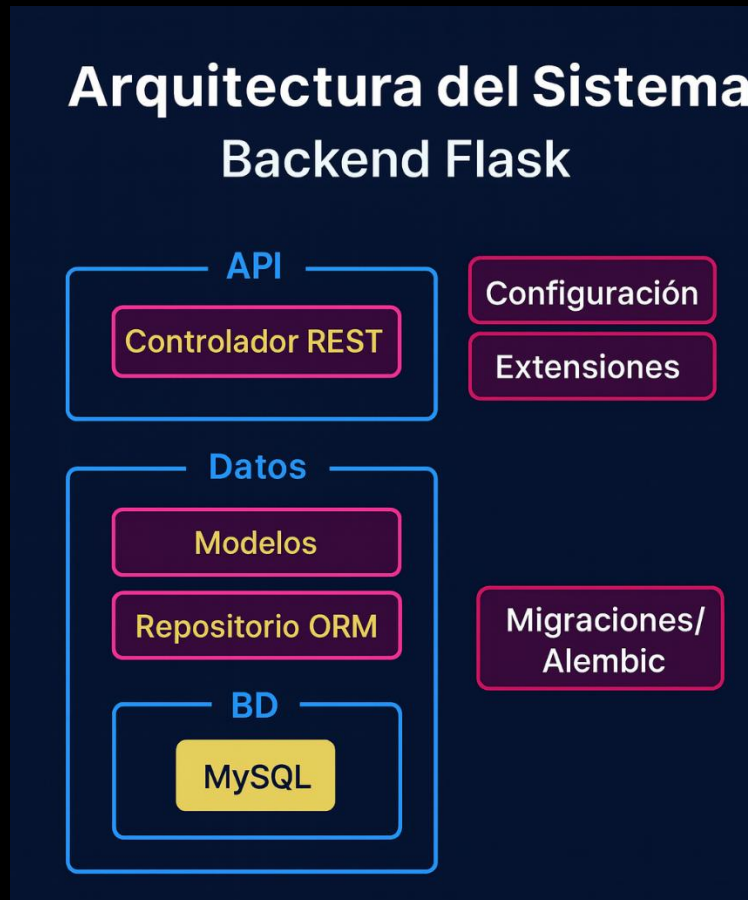


# Back End con Flask Rest



Aquí tienes una guía clara, paso a paso, para crear tu backend de **Recursos Humanos** con **Flask + MySQL** en **Windows** usando **PyCharm**, con **Flask-RESTful**, **SQLAlchemy**, **Flask-Migrate (Alembic)**, **PyMySQL**, **Marshmallow** y **CORS**. Incluye la estructura de archivos que indicaste y **todo el código completo** de cada archivo.

## 0) Requisitos previos

- Python 3.11 o 3.12 instalado.
- MySQL en localhost (con usuario y contraseña que conozcas).
- PyCharm (o tu editor favorito).
- Postman o `curl` para pruebas.

# 1) Crea el proyecto y el entorno virtual (Windows)

```
mkdir flask-rh-api
cd flask-rh-api

python -m venv .venv
.\.venv\Scripts\Activate.ps1    # (PowerShell)
# Si usas CMD: .\.venv\Scripts\activate.bat
```

---

# 2) Instala las dependencias

```
pip install Flask Flask-RESTful Flask-SQLAlchemy Flask-Migrate PyMySQL
Flask-Marshmallow marshmallow-sqlalchemy Flask-Cors
```

---

# 3) Crea la base de datos en MySQL (una sola vez)

**Nota:** Flask-Migrate (Alembic) maneja **tablas y cambios de esquema**, pero **no crea la base de datos**. Créala tú primero:

```
-- Conéctate a MySQL y ejecuta:
CREATE DATABASE recursos_humanos_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

---

# 4) Estructura del proyecto

Crea las carpetas y archivos siguientes:

```
flask-rh-api/
├─ app.py
├─ extensions.py
├─ models.py
├─ schemas.py
├─ api/
│  └─ __init__.py
│  └─ empleados.py
```

---

## 5) Código de cada archivo (con explicación breve)

### 5.1 `extensions.py`

**Por qué así:** Centralizamos las extensiones (SQLAlchemy, Migrate, Marshmallow) para evitar importaciones circulares y facilitar pruebas.

```
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
from flask_marshmallow import Marshmallow

db = SQLAlchemy()
migrate = Migrate()
ma = Marshmallow()
```

---

### 5.2 `models.py`

**Por qué así:** Definimos el modelo ORM con SQLAlchemy mapeado a la tabla requerida: `idEmpleado` (PK autoincremental), `nombre`, `departamento`, `sueldo` (Decimal 10,2).

```
from extensions import db
from sqlalchemy import Numeric

class Empleado(db.Model):
    __tablename__ = "empleado"

    idEmpleado = db.Column(db.Integer, primary_key=True,
autoincrement=True)
    nombre = db.Column(db.String(100), nullable=False)
    departamento = db.Column(db.String(100), nullable=False)
    sueldo = db.Column(Numeric(10, 2), nullable=False)

    def __repr__(self) -> str:
        return f"<Empleado {self.idEmpleado} - {self.nombre}>"
```

---

### 5.3 `schemas.py`

**Por qué así:** Con **Marshmallow** validamos entrada y serializamos salida. Usamos validaciones simples (longitud y rango) y devolvemos `sueldo` como número (float) para comodidad del front.

```
from extensions import ma, db
from marshmallow import fields, validate
from models import Empleado
```

```

class EmpleadoSchema(ma.SQLAlchemySchema):
    class Meta:
        model = Empleado
        load_instance = True
        sqla_session = db.session

    idEmpleado = ma.auto_field(dump_only=True)
    nombre = ma.auto_field(required=True, validate=validate.Length(min=1,
max=100))
    departamento = ma.auto_field(required=True,
validate=validate.Length(min=1, max=100))
    # Guardamos DECIMAL en BD, pero exponemos float en JSON para
    facilitar consumo
    sueldo = fields.Float(required=True, validate=validate.Range(min=0))

empleado_schema = EmpleadoSchema()
empleados_schema = EmpleadoSchema(many=True)

```

---

## 5.4 api/\_\_init\_\_.py

**Por qué así:** Usamos **Blueprint** para modularidad y **Flask-RESTful** para declarar recursos tipo REST de forma limpia.

```

from flask import Blueprint
from flask_restful import Api

bp = Blueprint("api", __name__, url_prefix="/api")
api = Api(bp)

```

---

## 5.5 api/empleados.py

**Por qué así:** Creamos endpoints REST para **listar/crear y consultar/actualizar/eliminar** empleados, usando Marshmallow para validar.

```

from flask import request
from flask_restful import Resource
from marshmallow import ValidationError

from extensions import db
from models import Empleado
from schemas import empleado_schema, empleados_schema
from . import api

class Empleados(Resource):
    # GET /api/empleados
    def get(self):
        empleados =
Empleado.query.order_by(Empleado.idEmpleado.asc()).all()
        return empleados_schema.dump(empleados), 200

```

```

# POST /api/empleados
def post(self):
    data = request.get_json()
    if not data:
        return {"message": "Cuerpo JSON requerido."}, 400
    try:
        empleado = empleado_schema.load(data)
    except ValidationError as err:
        return {"errors": err.messages}, 422

    db.session.add(empleado)
    db.session.commit()
    return empleado_schema.dump(empleado), 201

class EmpleadoDetail(Resource):
    # GET /api/empleados/<id>
    def get(self, id):
        empleado = Empleado.query.get_or_404(id)
        return empleado_schema.dump(empleado), 200

    # PUT /api/empleados/<id>
    def put(self, id):
        empleado = Empleado.query.get_or_404(id)
        data = request.get_json()
        if not data:
            return {"message": "Cuerpo JSON requerido."}, 400
        try:
            empleado = empleado_schema.load(data, instance=empleado,
partial=False)
        except ValidationError as err:
            return {"errors": err.messages}, 422

        db.session.commit()
        return empleado_schema.dump(empleado), 200

    # DELETE /api/empleados/<id>
    def delete(self, id):
        empleado = Empleado.query.get_or_404(id)
        db.session.delete(empleado)
        db.session.commit()
        return {"message": "Empleado eliminado"}, 200

# Registro de rutas en el Api del blueprint
api.add_resource(Empleados, "/empleados", endpoint="empleados")
api.add_resource(EmpleadoDetail, "/empleados/<int:id>",
endpoint="empleado_detail")

```

---

## 5.6 app.py

**Por qué así:** Usamos **application factory** `create_app()` para poder correr migraciones y pruebas fácilmente. Registramos CORS, extensiones y blueprint.

```

from flask import Flask
from flask_cors import CORS

from extensions import db, migrate, ma
from api import bp as api_bp
import api.empleados # asegura que las rutas se registren
import models # asegura que los modelos estén visibles para
Alembic/Flask-Migrate

def create_app():
    app = Flask(__name__)

    # Ajusta usuario/contraseña/host según tu entorno
    app.config["SQLALCHEMY_DATABASE_URI"] = (

"mysql+pymysql://root:admin@localhost/recursos_humanos_db?charset=utf8mb4
"

    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
    app.config["JSON_AS_ASCII"] = False

    # Extensiones
    db.init_app(app)
    migrate.init_app(app, db)
    ma.init_app(app)

    # CORS (abrimos para /api/*; ajusta origins según tu front)
    CORS(app, resources={r"/api/*": {"origins": "*"}})

    # Blueprints
    app.register_blueprint(api_bp)

    @app.get("/")
    def home():
        return "API de Recursos Humanos (Flask)  "

    return app

if __name__ == "__main__":
    create_app().run(debug=True)

```

## 6) Migraciones con Alembic (Flask-Migrate)

Con **factory pattern**, indica a Flask cómo crear la app.

### Opción A (variable de entorno)

```
set FLASK_APP=app:create_app
```

```
flask db init
flask db migrate -m "create empleado"
flask db upgrade
```

### Opción B (flag explícita)

```
flask --app app:create_app db init
flask --app app:create_app db migrate -m "create empleado"
flask --app app:create_app db upgrade
```

Si todo va bien, Alembic generará la tabla `empleado` con:

- `idEmpleado` (PK autoincremental)
- `nombre` (VARCHAR 100, NOT NULL)
- `departamento` (VARCHAR 100, NOT NULL)
- `sueldo` (DECIMAL(10,2), NOT NULL)

---

## 7) Ejecutar la API

```
flask --app app:create_app run
```

Servidor por defecto en: `http://127.0.0.1:5000/`

---

## 8) Probar endpoints

### Crear empleado (POST)

```
curl -X POST http://127.0.0.1:5000/api/empleados ^
-H "Content-Type: application/json" ^
-d '{"nombre":"Ana", "departamento":"TI", "sueldo":12500.50}'
```

### Listar empleados (GET)

```
curl http://127.0.0.1:5000/api/empleados
```

### Detalle (GET)

```
curl http://127.0.0.1:5000/api/empleados/1
```

### Actualizar (PUT)

```
curl -X PUT http://127.0.0.1:5000/api/empleados/1 ^
```

```
-H "Content-Type: application/json" ^  
-d "{\"nombre\":\"Ana Pérez\", \"departamento\":\"TI\",  
\"sueldo\":13000}"
```

## Eliminar (DELETE)

```
curl -X DELETE http://127.0.0.1:5000/api/empleados/1
```

---

## 9) Notas y tips rápidos

- **Credenciales:** cambia `root:admin` por tu usuario/contraseña reales de MySQL.
- **Validación:** Marshmallow ya valida longitud y que `sueldo >= 0`.
- **Estructura limpia:** separar `extensions.py`, `models.py`, `schemas.py` y `api/` evita acoplamiento y facilita crecer.
- **Migrations:** cuando cambies el modelo, ejecuta `flask db migrate -m "mensaje"` y luego `flask db upgrade`.

Con esto tienes un backend **listo y estructurado** para tu app de Recursos Humanos

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)